

Lista de Exercícios: Recursividade em C++

Nível Básico: Entendendo a Lógica

O objetivo aqui é se acostumar com a estrutura de uma função recursiva: caso base e passo recursivo.

1. Contagem Regressiva

- **Tarefa:** Crie um procedimento recursivo `contagemRegressiva(int n)` que receba um número inteiro `n` e imprima todos os números de `n` até 1.
- **Exemplo de Chamada:** `contagemRegressiva(5);`

Saída Esperada:

5
4
3
2
1
•

2. Soma dos Primeiros N Números

- **Tarefa:** Implemente uma função recursiva `somaAteN(int n)` que calcule a soma de todos os números inteiros de 1 até `n`.
- **Fórmula:** $\$soma(n) = n + soma(n-1)\$$
- **Exemplo de Chamada:** `somaAteN(4);`
- **Saída Esperada:** 10 (pois $4 + 3 + 2 + 1 = 10$)

3. Potência

- **Tarefa:** Escreva uma função recursiva `potencia(int base, int expoente)` que calcule `base` elevada à potência `expoente`. Considere que o expoente será sempre um número inteiro não negativo.
- **Fórmula:** $\$potencia(b, e) = b * potencia(b, e-1)\$$
- **Exemplo de Chamada:** `potencia(2, 3);`
- **Saída Esperada:** 8 (pois $2^3 = 8$)

Nível Intermediário: Vetores e Strings

Aqui, aplicamos a recursividade para percorrer estruturas de dados.

4. Soma dos Elementos de um Vetor

- **Tarefa:** Crie uma função recursiva `somarVetor(int vetor[], int tamanho)` que receba um vetor de inteiros e seu tamanho, e retorne a soma de todos os seus elementos.
- **Dica:** O passo recursivo pode ser pensar na soma como `vetor[tamanho-1] + somarVetor(vetor, tamanho-1)`.

5. Impressão Inversa de uma String

- **Tarefa:** Desenvolva um procedimento recursivo `imprimirInverso(string s)` que receba uma string e a imprima na tela de trás para frente.
- **Dica:** Imprima o último caractere da string e depois chame a função para a substring que exclui esse último caractere.

6. Palíndromo

- **Tarefa:** Implemente uma função recursiva `ehPalindromo(string s, int inicio, int fim)` que retorne `true` se a string for um palíndromo (lê-se da mesma forma de trás para frente) e `false` caso contrário.
- **Dica:** O caso base ocorre quando os índices `inicio` e `fim` se cruzam. O passo recursivo compara os caracteres nas posições `inicio` e `fim` e chama a função para `inicio+1` e `fim-1`.

Nível Avançado: Algoritmos Clássicos

Estes são problemas mais complexos onde a solução recursiva é notoriamente elegante.

7. Sequência de Fibonacci

- **Tarefa:** Escreva uma função recursiva `fibonacci(int n)` que retorne o n-ésimo termo da sequência de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...).
- **Fórmula:** $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$
- **Casos Base:** `fib(0) = 0` e `fib(1) = 1`.

8. Busca Binária Recursiva

- **Tarefa:** Implemente uma função de busca binária recursiva `buscaBinaria(int vetor[], int alvo, int inicio, int fim)` que procure por um número `alvo` em um **vetor ordenado**. A função deve retornar o índice do elemento se encontrado, ou -1 caso contrário.
- **Lógica:** Verifique o elemento do meio. Se for o alvo, retorne o índice. Se o alvo for menor, chame a função recursivamente para a metade esquerda. Se for maior, chame para a metade direita.

9. Torre de Hanói

- **Tarefa:** Este é um quebra-cabeça clássico. Crie um procedimento recursivo `torreDeHanoi(int n, char origem, char destino, char auxiliar)` que imprima os passos necessários para mover `n` discos da torre `origem` para a torre `destino`, usando uma torre `auxiliar`.
- **Regras:**
 1. Mover `n-1` discos da `origem` para a `auxiliar`.
 2. Mover o disco `n` (o maior) da `origem` para o `destino`.
 3. Mover os `n-1` discos da `auxiliar` para o `destino`.
- **Exemplo de Chamada:** `torreDeHanoi(3, 'A', 'C', 'B');`